< Go to the original

Capability	AWS	Azure	GCP	Open-Source
Agent Identity	AgentCore Identity (ARN-based workload IDs)	Entra Agent ID (Entra ID principals)	Agent Engine Identity (SPIFFE-based IAM principals)	SPIFFE/SPIRE + K8s Service Accounts + Keycloak
Managed Identity	IAM Roles Anywhere + AgentCore Identity	Azure Managed Identity + Entra WIF	Cloud IAM Managed Workload Identity (SPIFFE-native)	SPIFFE/SPIRE (CNCF graduated) + Istio mTLS
Sponsorship	Workload access tokens (user-to-agent binding)	Entra ID Governance sponsors (human accountability)	IAM service account impersonation (delegation chain)	Keycloak delegated auth + OPA policies
Access Packages	IAM permission boundaries + Cognito scopes	Entra Entitlement Management (time-limited)	Cloud IAM conditions + Apigee API products	Keycloak UMA 2.0 + OPA
Credential Mgmt	Secrets Manager + KMS + AgentCore token vault	Key Vault + Managed Identity (credential-free)	Secret Manager + Cloud KMS + short-lived X.509	HashiCorp Vault + SPIRE auto-rotation + SOPS

Agentic Identity in AWS, Azure, GCP and OSS: Workloads, Sponsorship, Secrets, and Shadow Detection

Freedium ko-fi librepay patreon

ids

Agent Native

Follow

androidstudio · March 6, 2026 (Updated: March 6, 2026) · Free: No

The single biggest blind spot in most teams' architecture is identity management: **who is this agent, what is it allowed to do, and who is accountable when it does something it should not?**

Over the past few months, all three major cloud providers, AWS, Azure, and GCP have shipped first-class agent identity primitives, almost simultaneously.

AWS launched Amazon Bedrock AgentCore Identity, Microsoft introduced Entra Agent ID and Google Cloud introduced Vertex AI Agent Engine's agent identities.

And the open-source world has SPIFFE/SPIRE, a CNCF-graduated standard, increasingly positioned as the identity backbone for non-human workloads including AI agents.

In this article, I will share what I have learned after digging into each of these systems, what they actually do, where they converge, and where the gaps are.

If you are building anything with autonomous agents in production, this is the infrastructure layer you should not ignore.

Why Is Agent Identity Suddenly Everywhere?

Agents are no longer stateless function calls.

They persist, they delegate, they make decisions, and they touch systems on behalf of real users.

That practically changes everything about how identity works.

As recently as mid-2025, most teams I talked to were still assigning agents

shared service accounts or, worse, hardcoded API keys.

Google's Cybersecurity 2026 Forecast Report explicitly warns of a "shadow agent" crisis referring to AI agents deployed by employees without corporate oversight, creating invisible pipelines for sensitive data.

Most teams have implemented or are actively scaling AI agents, yet only a few having comprehensive AI-specific security controls in place.

The old model, where an agent is just another microservice with a service account breaks down the moment agents start acting autonomously, spawning sub-agents, or operating across organizational boundaries.

And agent identity is not just "IAM for bots", it is a new identity category that sits alongside human identities and traditional workload identities, with its own authentication patterns, delegation models, and lifecycle states.

> Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.

> **You can read it here:** ***Agentic SaaS Patterns Winning in 2026****,* packed with real-world examples, architectures, and workflows you won't find anywhere else.

OK.. What Exactly Is Agentic Identity?

An agentic identity is a durable, unique, cryptographically verifiable identity assigned to an AI agent that allows it to authenticate, be authorized, be audited, and be governed, independently from the human user or system that created it.

Before these new primitives, an agent typically inherited its creator's identity (a service account, an API key, a user token).

The agent was invisible in audit logs. You could not write a policy that said "this specific agent can read from this table but not that one."

You definitely could not answer the question "which human authorized this agent to do what it just did?"

This means that getting identity wrong at the agent layer creates a class of security and compliance failures that are qualitatively different from traditional IAM gaps.

When a human user oversteps their permissions, you have an audit trail pointing to a person.

When an unidentified agent does it, you often have nothing.

The Pillars of Agent Identity

There are five capabilities that every provider now addresses:

Capability	What It Does	Why It Matters
Workload / Agent Identity	Assigns each agent a unique, persistent identity (ARN, principal URI, SPIFFE ID)	Agents become first-class auditable entities, not anonymous processes
Managed Identity / SPIFFE	Issues short-lived, automatically rotated credentials tied to the agent's workload identity	Eliminates hardcoded secrets; enables zero-trust agent-to-agent communication
Agent Sponsorship Model	Binds every agent action to a human sponsor who is accountable	Creates a chain of delegation: human → agent → action
Access Packages / Scopes	Defines which APIs, data, and services the agent can touch, with time-limited grants	Enforces least privilege with granularity beyond what service accounts allow
Credential Posture / Rotation	Manages, rotates, and vaults all agent credentials automatically	Reduces the attack surface of long-lived tokens and API keys

Most teams I have worked with get one or two of these right (usually the identity and the credentials) but completely skip the sponsorship model and the access packages, which is where the real risk accumulates.

How Does Each Cloud Provider Implement This?

Each provider has taken a meaningfully different architectural approach, and the differences matter for how you design your agent platform.

AWS: Amazon Bedrock AgentCore Identity

AWS shipped AgentCore Identity as part of the broader Bedrock AgentCore platform. I think the design is notable for how tightly it integrates identity into the agent runtime itself.

Every agent deployed to AgentCore Runtime automatically receives a unique ARN-based workload identity. The system has four core components:

- Agent Identity Directory (centralized registry),

- Agent Authorizer (validates inbound requests),
- Resource Credential Provider (stores access configurations),
- Resource Token Vault (securely holds OAuth tokens and API keys).

Inbound authentication verifies who is calling the agent (using JWT, OAuth 2.0, or SigV4).

Outbound authentication manages how the agent authenticates to downstream resources.

This separation means a compromised inbound token cannot be reused to access downstream systems.

Sponsorship implementation

AgentCore Identity uses "workload access tokens" that bind a specific user to a specific agent invocation.

The agent cannot exceed the delegated scope of the sponsoring user. Resource credential providers store the OAuth tokens the agent is allowed to use, all scoped to the consenting user.

Credential management

IAM role temporary credentials expire after 1 hour by default (up to 12 hours), token vault handles OAuth refresh automatically and AWS Secrets Manager provides auto-rotation for API keys and database passwords.

Azure: Microsoft Entra Agent ID

Entra Agent ID assigns every AI agent a durable digital identity within Entra ID.

This is the same identity fabric that manages human users, meaning agents show up in sign-in logs, are subject to Conditional Access policies, and can be governed through the same admin portals.

New RBAC roles (Agent ID Administrator, Agent ID Developer, Agent Registry Administrator) provide granular administrative control.

Sponsorship implementation

This is where Azure is arguably ahead of the pack. Entra Agent ID Governance explicitly introduces the concept of "sponsors", human users who are accountable for an agent's identity lifecycle and access decisions.

Sponsors receive notifications about access expiry.

When a sponsor leaves the organization, Lifecycle Workflows automatically transfer sponsorship to their manager.

Access packages

Entra Entitlement Management extends access packages to agent identities with configurable expiry dates, approval workflows, and reviewers. Time-limited grants trigger re-approval cycles before expiration.

Shadow agent detection

Microsoft Defender for Cloud's AI Security Posture Management discovers AI assets and agent deployments across Azure, AWS, and GCP, identifying unauthorized agents.

This multi-cloud detection capability is a significant differentiator.

GCP: Vertex AI Agent Engine + Cloud IAM

Google's approach is interesting because it builds directly on SPIFFE, the open identity standard, rather than creating a proprietary identity primitive.

Vertex AI Agent Engine provisions a unique IAM principal for each deployed agent.

The identity is based on SPIFFE, and Google auto-provisions and manages an X.509 certificate for authentication.

I think this is very novel given that it bridges the gap between cloud-native identity and open standards.

If you are already using SPIFFE/SPIRE for your non-agent workloads, GCP's approach means your agent identities will speak the same protocol as everything else in your mesh.

Managed Identity / SPIFFE

Cloud IAM Managed Workload Identity provides SPIFFE-compliant identities for workloads on GKE and Compute Engine, issuing SVIDs as X.509 certificates with mTLS.

Workload Identity Federation extends this to external workloads via OIDC/SAML.

Shadow agent detection

Security Command Center AI Protection automatically discovers and catalogs all AI assets within a GCP organization.

Cloud Asset Inventory provides a comprehensive searchable inventory.

Open-Source: SPIFFE/SPIRE + Keycloak + OPA + Vault

The open-source stack for agent identity is actually remarkably complete, if you are willing to assemble the pieces yourself.

Workload identity

SPIFFE/SPIRE (CNCF graduated) issues short-lived X.509 SVIDs and JWT tokens based on workload attestation.

Keycloak service accounts provide OAuth2 client credentials flow for agents.

Sponsorship

Keycloak's token exchange and delegated authorization enables a sponsorship model where a human user delegates specific permissions to an agent for a scoped duration.

OPA policies enforce which agents are sponsored by which users and

what actions are permitted.

Credential management

HashiCorp Vault manages dynamic secrets with automatic rotation. SPIFFE/SPIRE automatically rotates SVIDs before expiry. SOPS encrypts secrets in Git.

Shadow agent detection

Falco detects unauthorized agent processes at runtime via eBPF kernel monitoring. OPA Gatekeeper enforces admission policies preventing unregistered agent deployments. Cilium provides deep network observability.

Whether the OSS path works for you depends entirely on your team's operational capacity.

The tools are mature and production-tested individually. The integration burden is real but manageable if you already run a Kubernetes-native platform.

Cross-Cloud Comparison: Agent Identity Capabilities

Let's have a look at each of the five Identity & Entitlements capabilities per specific services across all four ecosystems.

Capability	AWS	Azure	GCP	Open-Source
Agent Identity	AgentCore Identity (ARN-based workload IDs)	Entra Agent ID (Entra ID principals)	Agent Engine Identity (SPIFFE-based IAM principals)	SPIFFE/SPIRE + K8s Service Accounts + Keycloak
Managed Identity	IAM Roles Anywhere + AgentCore Identity	Azure Managed Identity + Entra WIF	Cloud IAM Managed Workload Identity (SPIFFE-native)	SPIFFE/SPIRE (CNCF graduated) + Istio mTLS
Sponsorship	Workload access tokens (user-to-agent binding)	Entra ID Governance sponsors (human accountability)	IAM service account impersonation (delegation chain)	Keycloak delegated auth + OPA policies
Access Packages	IAM permission boundaries + Cognito scopes	Entra Entitlement Management (time-limited)	Cloud IAM conditions + Apigee API products	Keycloak UMA 2.0 + OPA
Credential Mgmt	Secrets Manager + KMS + AgentCore token vault	Key Vault + Managed Identity (credential-free)	Secret Manager + Cloud KMS + short-lived X.509	HashiCorp Vault + SPIRE auto-rotation + SOPS

No single provider has a perfect story across all five capabilities:

- Azure's sponsorship model is the most mature
- AWS's integrated token vault is the tightest for Bedrock workloads
- GCP's SPIFFE-native approach is the most portable
- And the open-source stack gives you the most control at the cost of integration effort

What About Agent Lifecycle Management?

Identity is only half the story.

The other half is lifecycle: how does an agent go from an idea to a registered, approved, actively running production workload, and how do you deprecate, revoke, or quarantine it when things go wrong?

Register >> Approve >> Active:

Provider	Register → Approve → Active Implementation
AWS	Bedrock Agents: create (draft) → test sandbox → create version → alias for production. SageMaker Model Registry: Pending/Approved/Rejected gates.
Azure	Agents get Entra Agent IDs on creation. Deployment from Foundry to M365 routes through Admin Center for IT approval. AI Gateway centralizes inventory.
GCP	Registration in Agent Engine (provisions identity), approval via Cloud Deploy (manual gates), activation in Gemini Enterprise Agent Gallery.
OSS	Backstage catalog + MLflow stage transitions (Staging → Production) + ArgoCD promotion gates.

This is very helpful especially when I am working on multi-team platforms where different groups build agents independently.

Having a formal approval gate before an agent goes live, and having that gate tied to the agent's identity means you can enforce governance without bottlenecking every team.

Deprecate, Revoke and Quarantine

Most teams plan for agent creation but not for agent teardown.

You should treat deprecation as a first-class capability:

- AWS Bedrock Agent aliases redirect traffic from deprecated versions. IAM revocation + AgentCore Policy for emergency quarantine.
- Azure Entra Agent ID Governance lets sponsors disable agents. Microsoft Defender quarantines agents posing security or compliance risks.
- GCP IAM permission revocation immediately blocks all resource access. VPC Service Controls remove agents from access perimeters.
- OSS Keycloak session revocation + Kubernetes network policies for

quarantine + OPA enforcement of "revoked" status.

Shadow Agent Detection

If you have been deploying agents and wondering why your security team keeps asking about "unauthorized AI workloads," you are not doing anything wrong, it is a real and growing problem.

Unsanctioned AI tool usage grew significantly (as much as 70%) in 2025:

Here are some mitigation services:

- AWS CloudTrail + GuardDuty Extended Threat Detection with ML-based anomaly detection.
- Azure Defender for Cloud AI Security Posture Management discovers agents across Azure, AWS, and GCP (multi-cloud).
- GCP Security Command Center AI Protection auto-catalogs all AI assets in a GCP organization.
- OSS Falco (eBPF runtime) + OPA Gatekeeper (admission control) + Cilium (network observability).

Things to Pay Attention

Sponsorship Is the Missing Link

Agent identity without sponsorship is identity without accountability.

If your current agent setup cannot answer "which human authorized this agent to take this action," you have a governance gap.

2. SPIFFE Is the Cross-Cloud Identity Standard

I ran into this exact realization myself before I fully understood GCP's approach. Google's agent identities are SPIFFE-native. HashiCorp Vault Enterprise added native SPIFFE authentication in its 1.21 release. SPIFFE Federation allows workloads across different trust domains to authenticate each other. For multi-cloud agent deployments, SPIFFE is the closest thing to a lingua franca we have.

3. Shadow Agent Detection Is No Longer Optional

The attack surface is expanding faster than governance. Every provider now ships shadow agent discovery, use it.

4. The OSS Stack Is Production-Ready

For teams already running Kubernetes with SPIFFE/SPIRE, Keycloak, OPA, and Vault, the open-source agent identity stack is genuinely mature.

SPIFFE is CNCF graduated. Falco provides eBPF-level runtime detection. The cost is operational, not technical.

5. Token Vaults Complete the Credential Story

Auto-rotating secrets in Vault or Secrets Manager is necessary but not sufficient. Agents also hold OAuth tokens, API keys from third-party services, and session tokens that need automated refresh. AWS's AgentCore token vault and Azure's Managed Identity (credential-free for Azure-hosted agents) are the patterns to follow.

Limitations and Open Questions

It is very easy to get excited about new infrastructure capabilities and forget the gaps.

- **Semantic identity is unsolved.** SPIFFE proves "this agent process came from my orchestrator." It does not prove "this agent is aligned with governance policy X." Those higher-level attestation claims require a layer above workload identity that nobody has fully built yet.
- **Cross-cloud federation is early.** Production-grade multi-cloud agent identity federation is still emerging (e.g. where an agent on AWS proves its identity to a service on GCP with full audit trail). I suspect the answer is SPIFFE-based, but this needs more testing.
- **Lifecycle automation is patchy.** Register-approve-activate workflows exist, but they are often disconnected from the identity system. Revoking an agent's identity and revoking its deployment are separate operations on every platform.
- **Entitlement management is still coarse.** Access packages scope what APIs an agent can call, but they rarely scope what data within those APIs the agent can see. Row-level or column-level data governance tied to agent identity is a gap across every provider.

Concluding Thoughts

Agent identity is production infrastructure that every major cloud provider has shipped, and the open-source ecosystem has mature alternatives.

The five-pillar model, i.e. workload identity, managed credentials, sponsorship, access packages, and credential posture gives you a checklist for evaluating your own agent platform's readiness.

If you are missing any of these, you have a gap that gets more expensive to fill the longer you wait.

If you are starting a new agent platform today, my concrete advice:

1. **Pick an identity primitive and commit.** AgentCore Identity, Entra Agent ID, Agent Engine Identity, or SPIFFE/SPIRE, any of them is better than shared service accounts.
2. **Implement sponsorship from day one.** Even if your provider does not enforce it natively, create a metadata field that records which human authorized each agent.
3. **Turn on shadow agent detection now.** Every cloud provider offers it. The cost of not knowing what agents are running is higher than the cost of monitoring.
4. **Plan for agent deprecation.** Your lifecycle model needs a kill switch.

Bonus Articles

7 Local LLM Families To Replace Claude/Codex (for everyday tasks)

Open-source model families you can run locally that are now delivering real-world performance surprisingly close to…

medium.com

These 4 Agents Ship Products and Print Revenue on Autopilot

FelixCraftAI made $76K selling digital products. JunoAgent has 50 paying members. KellyClaudeAI shipped 5 apps to the…

medium.com

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for…

medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD (“Get Shit Done”) aims to solve context rot, the quality degradation as the model’s context window fills.

medium.com

OpenClaw Memory Systems That Don’t Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it… it’s not random.

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

#aws #azure #google-cloud-platform #agentic-ai #ai-agent-security